



LẬP TRÌNH C

- ✓ Kiến thức cơ bản
- ✓ Khái niệm
- ✓ Chương trình
- ✓ Luyện tập

```
#include <stdio.h>
int main() {
    printf("Hello, C!");
    return 0;
}
```



Code Ý Tưởng
Của Bạn

>> Từ Cơ Bản Đến Nâng Cao <<

1. Nền Tảng Về C

★ C Là Gì?

C là ngôn ngữ lập trình đa năng, hướng thủ tục (procedural), được Dennis Ritchie phát triển năm 1972 tại Bell Labs. Đây là nền tảng của nhiều ngôn ngữ hiện đại như C++, Java, Python, v.v.

Đặc Điểm Của C

- Đơn giản và hiệu quả
- Portable (độc lập với máy)
- Ngôn ngữ bậc trung (mid-level)
- Lập trình có cấu trúc (structured programming)
- Bộ toán tử (operator) phong phú
- Thực thi nhanh

★ Cấu Trúc Chương Trình C

Phần Tài Liệu (Documentation Section)	→ // Chú thích (Comments)
Phần Liên Kết (Link Section)	→ // Header Files
Phần Định Nghĩa (Definition Section)	→ // Macro, hằng số (Constants)
Hàm main() { Statements; return 0; }	→ // Điểm bắt đầu của mọi chương trình C

★ Header Files

Header files là các chương trình được viết sẵn, chứa những hàm hữu ích. Ta include chúng bằng #include

Ví dụ:

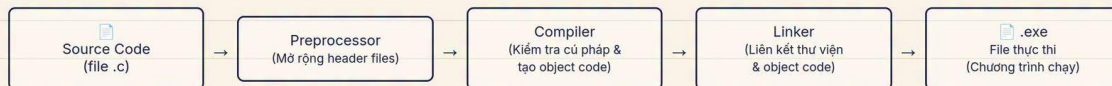
```
#include <stdio.h>    // Input/Output
#include <conio.h>     // Console I/O
#include <math.h>      // Các hàm Math
```

★ Hàm main()

- Mọi chương trình C đều phải có hàm main().
- Đây là điểm bắt đầu (entry point) khi chương trình thực thi.
- Chương trình bắt đầu chạy từ main() và kết thúc ở cuối main().

```
int main()           → Kiểu trả về (int)
{
  // code
  return 0;          → Thân hàm (Body)
}                    → Câu lệnh return
```

★ Biên Dịch & Thực Thi



★ Chương Trình C Đầu Tiên

```
#include <stdio.h>
int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

Giải Thích:

- #include <stdio.h> → thêm thư viện input-output chuẩn
- printf() → in text ra màn hình
- \n → xuống dòng
- return 0; → báo hiệu kết thúc thành công

Cố lên nào! 😊

LẬP TRÌNH C

2. BIẾN, KIỂU DỮ LIỆU & HẲNG SỐ

★ Biến (Variables)

Biến là một vùng nhớ có tên, dùng để lưu trữ dữ liệu có thể thay đổi trong quá trình chương trình thực thi.

Ví dụ:

```
int age = 20;           // khai báo + khởi tạo
float salary;           // khai báo
salary = 50000.50;      // khởi tạo
```

★ Hằng Số (Constants)

Hằng số là các giá trị cố định, không thay đổi trong suốt quá trình thực thi.

Ví dụ:

```
const float PI = 3.14159;
#define MAX 100
```

★ Kiểu Dữ Liệu Trong C

Kiểu	Mô Tả	Kích Thước (Bytes)	Phạm Vi (Xấp Xi)
char	Ký tự	1	-128 đến 127
int	Số nguyên	2 hoặc 4	-32,768 đến 32,767 hoặc -2^{31} đến $2^{31}-1$
short int	Số nguyên ngắn	2	-32,768 đến 32,767
long int	Số nguyên dài	4	-2^{31} đến $2^{31}-1$
long long int	Số nguyên rất dài	8	-2^{63} đến $2^{63}-1$
float	Số thực dấu phẩy động	4	$\pm 1.2E-38$ đến $\pm 3.4E+38$
double	Số thực độ chính xác kép	8	$\pm 2.3E-308$ đến $\pm 1.7E+308$
long double	Số thực dài	10 hoặc 16	Độ chính xác cao hơn
void	Không có giá trị	0	—

★ Type Modifiers

Dùng để điều chỉnh các kiểu dữ liệu cơ bản.

```
signed    → lưu được cả giá trị +/- (mặc định)
unsigned  → chỉ lưu giá trị 0 và +
short     → giảm kích thước
long      → tăng kích thước
```

★ Toán Tử sizeof()

Trả về kích thước (theo byte) của một kiểu dữ liệu hoặc biến.

Ví dụ:

```
printf("%lu", sizeof(int));    // 4
printf("%lu", sizeof(char));   // 1
printf("%lu", sizeof(float));  // 4
printf("%lu", sizeof(double)); // 8
```

★ Keywords Trong C (Ví Dụ)

int, if, else, for, while, return, char, float, double, void, break, continue, switch, case, default, struct, union, enum, v.v.

★ Identifiers (Quy Tắc Đặt Tên)

- Có thể chứa chữ cái, chữ số và dấu _
- Phải bắt đầu bằng chữ cái hoặc dấu _
- Không được bắt đầu bằng chữ số
- Không được chứa ký tự đặc biệt
- Phân biệt chữ hoa/chữ thường (case-sensitive)

Ví dụ:

```
✓ int age           ✓ _value
✓ total123          ✗ 1value
✗ my-value
```

★ Type Conversion

Chuyển đổi từ kiểu dữ liệu này sang kiểu khác.

- Implicit (Tự động)


```
int a = 10;
float b = a; // int sang float
```
- Explicit (Thủ công)


```
float x = 5.67;
int y = (int)x; // float sang int
```

💡 Lưu Ý Nhanh

- Chọn đúng kiểu dữ liệu để tiết kiệm bộ nhớ
- Dùng const cho các giá trị cố định
- Đặt tên biến có ý nghĩa

LẬP TRÌNH C

3. TOÁN TỬ & BIỂU THỨC

★ Toán Tử (Operators)

Toán tử là các ký hiệu đặc biệt dùng để thực hiện phép toán trên biến và giá trị. Biểu thức (expressions) là sự kết hợp giữa biến, giá trị và toán tử.

★ Các Loại Toán Tử Trong C

- 1. Toán tử Số học (Arithmetic)
- 5. Toán tử Bitwise
- 2. Toán tử Quan hệ (Relational)
- 6. Toán tử Tăng/Giảm (Increment/Decrement)
- 3. Toán tử Logic
- 7. Toán tử Ba Ngôi (Ternary/Conditional)
- 4. Toán tử Gán (Assignment)
- 8. Toán tử Đặc Biệt

★ 1. Toán Tử Số Học

Toán tử	Ý nghĩa	Ví dụ	Kết quả
+	Cộng	a + b	Tổng
-	Trừ	a - b	Hiệu
*	Nhân	a * b	Tích
/	Chia	a / b	Thương
%	Chia lấy dư	a % b	Số dư

Ví dụ:

```
int a = 17, b = 5;
a + b = 22    a - b = 12    a * b = 85
a / b = 3     a % b = 2
Lưu ý: Trong C, phép chia số nguyên (/)
chỉ trả về kết quả nguyên.
```

★ 2. Toán Tử Quan Hệ

Toán tử	Ý nghĩa	Ví dụ	Kết quả
==	Bằng	a == b	1 (true) / 0 (false)
!=	Khác	a != b	1 hoặc 0
>	Lớn hơn	a > b	1 hoặc 0
<	Nhỏ hơn	a < b	1 hoặc 0
>=	Lớn hơn hoặc bằng	a >= b	1 hoặc 0
<=	Nhỏ hơn hoặc bằng	a <= b	1 hoặc 0

Ví dụ:

```
int a = 10, b = 20;
a==b-0    a!=b-1    a>b-0
a<b-1    a>=b-0    a<=b-1
Trong C, toán tử quan hệ trả về
1 nếu true, 0 nếu false.
```

★ 3. Toán Tử Logic

Toán tử	Ý nghĩa	Ví dụ	Kết quả
&&	AND logic	(a>5 && b<10)	1 hoặc 0
	OR logic	(a>5 b<10)	1 hoặc 0
!	NOT logic	!(a > b)	1 hoặc 0

Ví dụ:

```
int a = 8, b = 12;
(a > 5 && b < 10) -> 0
(a > 5 || b < 10) -> 1
!(a > b) -> 1
```

Short-Circuiting:

- Trong &&, nếu điều kiện đầu false, điều kiện sau không được xét.
- Trong ||, nếu điều kiện đầu true, điều kiện sau không được xét.

★ 4. Toán Tử Gán

Toán tử	Ý nghĩa	Tương đương	Ví dụ
=	Gán	a = b	a = 10
+=	Cộng và gán	a = a + b	a += 5
-=	Trừ và gán	a = a - b	a -= 3
*=	Nhân và gán	a = a * b	a *= 2
/=	Chia và gán	a = a / b	a /= 4
%=	Chia dư và gán	a = a % b	a %= 3

Ví dụ:

```
int a = 10;
a += 5; // a=15    a -= 3; // a=12    a%=2; // a=24
a *= 2; // a=24    a /= 4; // a=6
```

★ 5. Toán Tử Bitwise

Toán tử	Ý nghĩa	Ví dụ	Kết quả
&	Bitwise AND	a & b	Bitwise AND
	Bitwise OR	a b	Bitwise OR
^	Bitwise XOR	a ^ b	Bitwise XOR
~	Bitwise NOT	~a	Bù 1 (1's complement)
<<	Dịch trái	a << 1	Dịch trái 1 bit
>>	Dịch phải	a >> 1	Dịch phải 1 bit

Ví dụ:

```
int a=5; // 0101
int b=3; // 0011
a&b=1 // 0001
a|b=7 // 0111
a^b=6 // 0110
~a=-6 // bù 2
a<<1=10 // 1010
a>>1=2 // 0010
```

★ 6. Toán Tử Tăng/Giảm

Toán tử	Ý nghĩa	Ví dụ
++	Tăng thêm 1	a++, ++a
--	Giảm đi 1	a--, --a

Post Increment/Decrement (Dùng trước, đổi sau)

```
int a = 5;
b = a++; // b=5, a=6
c = a--; // c=6, a=5
```

Pre Increment/Decrement (Đổi trước, dùng sau)

```
int a = 5;
b = ++a; // a=6, b=6
c = --a; // a=5, c=5
```



Mẹo Nhanh: Hãy luyện viết biểu thức bằng cách kết hợp nhiều toán tử khác nhau.

Ví dụ: (a + b) * c // số học a > b && c < d // logic a = b++ + 5 // kết hợp

LẬP TRÌNH C

4. CÂU LỆNH ĐIỀU KIỆN

★ Câu lệnh điều kiện dùng để thực thi một khối code dựa trên một điều kiện.
C cung cấp các câu lệnh điều kiện sau:

Các loại:

1. Câu lệnh if
2. Câu lệnh if-else
3. Chuỗi else-if (else-if ladder)
4. Câu lệnh if lồng nhau (nested if)
5. Câu lệnh switch

★ 1. Câu Lệnh if

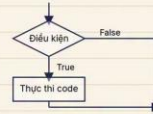
Thực thi một khối code nếu điều kiện đúng.

Cú pháp:

```
if (điều kiện)
{
    // code
}
```

Ví dụ:

```
int x = 10;
if (x > 0)
{
    printf("Dương");
}
```



★ 2. Câu Lệnh if-else

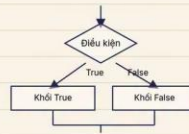
Thực thi một khối code nếu điều kiện đúng, ngược lại thực thi khối khác.

Cú pháp:

```
if (điều kiện)
{
    // code nếu true
}
else
{
    // code nếu false
}
```

Ví dụ:

```
int x = -5;
if (x >= 0)
{
    printf("Dương");
}
else
{
    printf("Âm");
}
```



★ 3. Chuỗi else-if

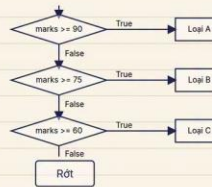
Dùng để kiểm tra nhiều điều kiện lần lượt.

Cú pháp:

```
if (điều_kiện_1)
{ // code }
else if (điều_kiện_2)
{ // code }
else
{ // code }
```

Ví dụ:

```
int marks = 72;
if (marks >= 90)
    printf("Xếp loại A");
else if (marks >= 75)
    printf("Xếp loại B");
else if (marks >= 60)
    printf("Xếp loại C");
else
    printf("Rớt");
```



★ 4. Nested if (if lồng nhau)

Một if hoặc else-if nằm bên trong một if/else khác.

Ví dụ:

```
int age = 20;
if (age >= 18)
{
    if (age >= 60)
        printf("Người cao tuổi");
    else
        printf("Người lớn");
}
else
{
    printf("Trẻ vị thành niên");
}
```

```
if (điều_kiện_1) -- If Ngoài
{
    if (điều_kiện_2) -- If Trong
    { ... }
    else
    { ... }
}
else -- else Ngoài
{ ... }
```

★ 5. Câu Lệnh switch

Dùng để chọn một trong nhiều khối code để thực thi.

Cú pháp:

```
switch (biểu_thức)
{
    case giá_trị_1:
        // code
        break;
    case giá_trị_2:
        // code
        break;
    ...
    default:
        // code
}
```

Ví dụ:

```
int day = 2;
switch (day)
{
    case 1: printf("T2"); break;
    case 2: printf("T3"); break;
    case 3: printf("T4"); break;
    ...
    default: printf("Không hợp lệ");
}
```

☆ Lưu ý:

- 0 nghĩa là false, khác 0 nghĩa là true.
- Dùng chuỗi else-if khi có nhiều khoảng điều kiện.
- Tránh lồng if quá sâu để dễ đọc hơn.

💡 Mẹo:

- Dùng break để tránh fall-through trong switch.
- Giữ điều kiện đơn giản và hiệu quả.
- Kiểm tra các edge case để kết quả tốt hơn.

LẬP TRÌNH C

5. VÒNG LẶP & CÂU LỆNH ĐIỀU KHIỂN

★ Vòng lặp (loops) dùng để thực thi một khối code lặp đi lặp lại cho đến khi thỏa mãn một điều kiện.

Các Loại Vòng Lặp Trong C

- Vòng lặp for
- Vòng lặp while
- Vòng lặp do-while

★ 1. Vòng Lặp for

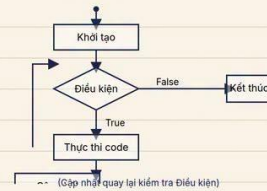
Dùng khi biết trước số lần lặp.

Cú pháp:

```
for (khởi_tạo; điều_kiện; cập_nhật)
{
    // code
}
```

Ví dụ:

```
int i;
for (i = 1; i ≤ 5; i++)
{
    printf("%d ", i);
}
// Kết quả: 1 2 3 4 5
```



★ 2. Vòng Lặp while

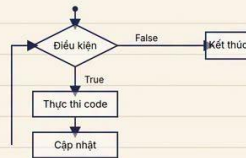
Dùng khi chưa biết trước số lần lặp.

Cú pháp:

```
while (điều_kiện)
{
    // code
    // cập nhật
}
```

Ví dụ:

```
int i = 1;
while (i ≤ 5)
{
    printf("%d ", i);
    i++;
}
// Kết quả: 1 2 3 4 5
```



★ 3. Vòng Lặp do-while

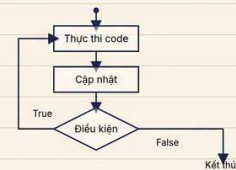
Tương tự while, nhưng code được thực thi ít nhất một lần.

Cú pháp:

```
do
{
    // code
    // cập nhật
} while (điều_kiện);
```

Ví dụ:

```
int i = 1;
do
{
    printf("%d ", i);
    i++;
} while (i ≤ 5);
// Kết quả: 1 2 3 4 5
```



★ 4. Câu Lệnh Điều Khiển

- **break** : Dừng vòng lặp hoặc switch.
- **continue** : Bỏ qua vòng lặp hiện tại, tiếp tục vòng kế.
- **goto** : Chuyển điều khiển đến một nhãn (label).
- **Vòng lặp vô hạn** : Khi điều kiện luôn đúng. (Cẩn thận khi dùng!)

Ví dụ: break và continue

```
for (int i = 1; i ≤ 5; i++)
{
    if (i == 3) continue; // bỏ qua 3
    if (i == 5) break; // dừng ở 5
    printf("%d ", i);
}
// Kết quả: 1 2 4
```

5. Vòng Lặp Lồng Nhau (Ví Dụ)

```
for (int i = 1; i ≤ 3; i++)
{
    for (int j = 1; j ≤ 3; j++)
    {
        printf("%d ", j);
    }
    printf("\n");
}
// Kết quả: 1 2 3 / 1 2 3 / 1 2 3
```

💡 Mẹo Nhanh

- Dùng for khi biết trước số lần lặp.
- Dùng while khi điều kiện phụ thuộc vào input người dùng hoặc số lần chưa xác định.
- Dùng do-while khi code cần chạy ít nhất một lần.

★ Các Mẫu Thường Gặp

- Đếm (1 đến N)
- Tính tổng (tổng của N số)
- Giai thừa (N!)
- Mẫu hình (dấu sao, số)

LẬP TRÌNH C

6. HÀM TRONG C

★ Hàm (function) là một khối code thực hiện một tác vụ cụ thể. Sử dụng hàm giúp code có tính module hóa, tái sử dụng được và dễ bảo trì.

★ Vì Sao Cần Dùng Hàm?

- ✓ Tránh lặp code
- ✓ Cải thiện khả năng đọc
- ✓ Dễ kiểm thử & debug
- ✓ Chia nhỏ vấn đề phức tạp thành các phần nhỏ hơn

★ Cấu Tạo Của Một Hàm

```
kiểu_trả_về tên_hàm(tham_số)
{
    // khai báo cục bộ
    // câu lệnh

    return giá_trị; (nếu có)
}
```

→ Function Header
→ Function Body
→ Return Statement

★ 1. Khai Báo Hàm (Prototype)

Cho compiler biết về hàm.

Cú pháp:

kiểu_trả_về tên_hàm(kiểu1 tham1, kiểu2 tham2, ...);

Ví dụ:

```
int add(int a, int b); // khai báo hàm
```

★ 2. Định Nghĩa Hàm (Function Definition)

Phản triển khai thực tế của hàm.

Ví dụ:

```
int add(int a, int b) // định nghĩa hàm
{
    // bắt đầu thân hàm
    int sum = a + b; // thực hiện xử lý
    return sum;      // trả kết quả
} // kết thúc thân hàm
```

★ 3. Gọi Hàm (Function Call)

Chuyển quyền điều khiển đến hàm được gọi.

Ví dụ:

```
int main()
{
    int x = 10, y = 20;
    int result = add(x, y); // gọi hàm
    printf("Sum = %d", result);
    return 0;
}
```

★ 4. Các Loại Hàm Trong C

Loại	Mô tả	Ví dụ
1. Không tham số, không trả về	Không nhận input, không xuất output	void hello(void)
2. Có tham số, không trả về	Nhận input, không xuất output	void display(int a)
3. Không tham số, có trả về	Không nhận input, trả về output	int getNumber(void)
4. Có tham số, có trả về	Nhận input, trả về output	int add(int a, int b)

1. Không tham số, không trả về

```
void hello(void)
{
    printf("Hello C!\n");
}

int main()
{
    hello(); // gọi
    return 0;
}
```

Kết quả:

Hello C!

2. Có tham số, không trả về

```
void display(int n)
{
    printf("Số = %d", n);
}

int main()
{
    display(25); // gọi
    return 0;
}
```

Kết quả:

Số = 25

3. Không tham số, có trả về

```
int getValue(void)
{
    return 100;
}

int main()
{
    int x = getValue();
    printf("GT=%d", x);
    return 0;
}
```

Kết quả:

Giá trị = 100

4. Có tham số, có trả về

```
int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int p = multiply(4, 5);
    printf("Tích=%d", p);
    return 0;
}
```

Kết quả:

Tích = 20

💡 Mẹo Nhanh

- Luôn khai báo hàm trước khi dùng (hoặc dùng prototype).
- Đặt tên hàm có ý nghĩa.
- Giữ hàm nhỏ gọn, tập trung vào một tác vụ.
- Dùng câu lệnh return để trả kết quả về nơi gọi.

⚠️ Lỗi Thường Gặp

- Quên khai báo kiểu trả về.
- Không return giá trị trong hàm non-void.
- Sai số lượng hoặc kiểu của tham số.
- Thiếu function prototype.

LẬP TRÌNH C

7. MẢNG TRONG C

★ Mảng (array) là tập hợp các phần tử cùng kiểu dữ liệu, được lưu trữ tại các vị trí bộ nhớ liên tiếp (contiguous).

Vi Sao Cần Mảng?

- Lưu nhiều giá trị cùng kiểu
- Dễ truy cập bằng chỉ số (index)
- Duyệt & xử lý hiệu quả

★ 1. Khai Báo

Cú pháp:

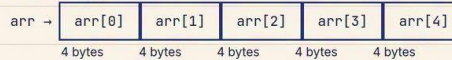
kiểu_dữ_liệu tên_mảng[kích_thước];

Ví dụ:

```
int arr[5]; // mảng 5 số nguyên
```

★ Biểu Diễn Trong Bộ Nhớ

Ví dụ: int arr[5];



Chỉ số bắt đầu từ 0 đến size-1

★ 2. Khởi Tạo

Cú pháp:

kiểu_dữ_liệu tên_mảng[size] = {val1, ..., valN};

Ví dụ:

```
int arr[5] = {10, 20, 30, 40, 50};  
// hoặc  
int arr[] = {10, 20, 30, 40, 50}; // size tùy chọn
```

★ 3. Truy Cập Phần Tử

Cú pháp:

tên_mảng[index]; // $0 \leq \text{index} < \text{size}$

Ví dụ:

```
int x = arr[2]; // truy cập phần tử thứ 3 (30)  
arr[1] = 25; // cập nhật phần tử thứ 2
```

★ 4. Duyệt Mảng (Traversal)

Ví dụ: In tất cả phần tử của mảng

```
int arr[5] = {10, 20, 30, 40, 50};  
int i;  
for (i = 0; i < 5; i++)  
{  
    printf("%d ", arr[i]);  
}  
// Kết quả: 10 20 30 40 50
```

★ 5. Mảng Với Input Người Dùng

Ví dụ: Đọc N số và lưu vào mảng

```
int n, i, arr[100];  
printf("Nhập kích thước mảng: ");  
scanf("%d", &n);  
for (i = 0; i < n; i++)  
{  
    printf("Nhập phần tử %d: ", i);  
    scanf("%d", &arr[i]);  
}
```

★ 6. Mảng Đa Chiều

Cú pháp:

kiểu_dữ_liệu tên_mảng[hàng][cột];

Ví dụ:

```
int matrix[3][4]; // 3 hàng, 4 cột
```

hàng 0	matrix[0][0]	matrix[0][1]	matrix[0][2]	matrix[0][3]
hàng 1	matrix[1][0]	matrix[1][1]	matrix[1][2]	matrix[1][3]
hàng 2	matrix[2][0]	matrix[2][1]	matrix[2][2]	matrix[2][3]

★ 7. Điểm Cần Nhớ

- Chỉ số mảng luôn bắt đầu từ 0.
- Truy cập chỉ số nằm ngoài phạm vi (out of bound) gây undefined behavior.
- Kích thước mảng phải cố định (không thể tăng khi runtime).
- Mảng được truyền vào hàm dưới dạng con trỏ (pointer).
- Tên mảng đại diện cho địa chỉ cơ sở (base address) của phần tử đầu tiên.

💡 **Pro Tip:** Luôn kiểm tra giới hạn mảng trước khi truy cập phần tử. Dùng tên biến có ý nghĩa và kích thước hợp lý.

★ **Luyện Tập Là Chìa Khóa!** Hãy thử viết các chương trình cơ sở sử dụng mảng.

LẬP TRÌNH C

8. CHUỖI TRONG C

★ Chuỗi (string) là một dãy ký tự, kết thúc bằng ký tự null '\0'.

Điểm Cần Nhớ

- String là một mảng ký tự (array of characters).
- Ký tự cuối phải là '\0'.
- Độ dài chuỗi = số ký tự trước '\0'.

★ 1. Khai Báo & Khởi Tạo

Cú pháp:

```
char str[size];
```

Ví dụ:

```
char str[10];           // khai báo
char name[20] = "Aman"; // khởi tạo
char msg[] = "Hello C"; // size tự động (8 kể cả \0)
char city[10] = {'D','e','l','h','i','\0'}; // từng ký tự
```

★ Biểu Diễn Trong Bộ Nhớ

Ví dụ: char str[] = "HELLO";

H	E	L	L	O	\0
48	45	4C	4C	4F	00

Ký tự null '\0' đánh dấu kết thúc chuỗi.

★ 2. Nhập & Xuất

Nhập (Input):

```
scanf("%s", str);
gets(str); // không khuyến khích (không an toàn)
```

Xuất (Output):

```
printf("%s", str);
puts(str); // in chuỗi kèm xuống dòng
```

★ 3. Độ Dài Chuỗi

Hàm: strlen() (trong <string.h>)

Cú pháp:

```
int length = strlen(str);
```

Ví dụ:

```
char str[] = "Crafted";
printf("Length = %d", strlen(str)); // KQ: 7
```

★ 4. Các Hàm Chuỗi Thường Dùng

Hàm	Mô tả	Ví dụ
strlen(str)	Trả về độ dài chuỗi	strlen("hello") // 5
strcpy(dest, src)	Sao chép src vào dest	strcpy(a, b);
strcat(dest, src)	Nối src vào cuối dest	strcat(a, b);
strcmp(str1, str2)	So sánh hai chuỗi	strcmp(a, b);
strrev(str)	Đảo ngược chuỗi	strrev(a);

★ 5. Chương Trình Ví Dụ

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[20] = "Crafted";
    char str2[20] = "_Note";

    printf("str1 = %s\n", str1);
    printf("Length = %d\n", strlen(str1));

    strcat(str1, str2); // Crafted_Note
    printf("After strcat: %s\n", str1);
    return 0;
}

// Kết quả:
// str1 = Crafted
// Length = 7
// After strcat: Crafted_Note
```

★ 6. So Sánh Chuỗi (strcmp)

Trả về:

```
< 0   nếu str1 < str2
0     nếu str1 == str2
> 0   nếu str1 > str2
```

Ví dụ:

```
strcmp("apple", "banana"); // < 0
strcmp("test", "test");    // 0
strcmp("zoo", "ant");      // > 0
```

💡 Mẹo Nhanh

- Luôn cấp đủ một byte cho '\0'.
- Ưu tiên dùng fgets() thay vì gets() để nhập an toàn.
- Dùng strlen() để tránh tràn bộ đệm (buffer overflow).

⚠️ Lỗi Thường Gặp

- Không chứa chỗ cho '\0'.
- Dùng gets() (không an toàn).
- Nhầm lẫn giá trị trả về của strcmp().
- Tràn bộ đệm do dư ký tự.

LẬP TRÌNH C

9. CON TRỎ TRONG C

★ Con trỏ (pointer) là một biến lưu địa chỉ của một biến khác.

Vì Sao Cần Con Trỏ?

- Sử dụng bộ nhớ hiệu quả
- Truyền dữ liệu lớn vào hàm
- Xây dựng cấu trúc dữ liệu động
- Truy cập phần tử mảng hiệu quả

★ 1. Khai Báo & Khởi Tạo

Cú pháp:

kiểu_dữ_liệu *tên_con_trỏ;

Ví dụ:

```
int *ptr; // con trỏ trỏ đến int
int a = 10;
ptr = &a; // ptr lưu địa chỉ của a
```

★ 2. Địa Chỉ & Dereference

& : toán tử lấy địa chỉ (address-of)
* : dereference (giá trị tại địa chỉ)

Ví dụ:

```
int a = 25;
int *ptr = &a;
printf("Địa chỉ a: %p\n", ptr);
printf("Giá trị a: %d\n", *ptr);
```

Biến	Địa chỉ	Giá trị
a	1000	25
ptr	2000	1000

ptr lưu địa chỉ 1000 của 'a'.

★ 3. Con Trỏ Với Các Kiểu Dữ Liệu

Kiểu DL	Kiểu con trỏ	Ví dụ
int	int *ptr	int a=5; int *ptr=&a;
char	char *ptr	char ch='A'; char *ptr=&ch;
float	float *ptr	float f=3.14; float *ptr=&f;
double	double *ptr	double d=2.718; double *ptr=&d;

★ 4. Số Học Con Trỏ (Pointer Arithmetic)

Với int *ptr, nếu ptr có địa chỉ A:

- ptr + 1 → A + sizeof(int)
- ptr - 1 → A - sizeof(int)
- ptr + n → A + n*sizeof(int)
- ptr - n → A - n*sizeof(int)

Ví dụ:

```
int arr[3] = {10, 20, 30};
int *p = arr;
printf("%d %d %d\n", *p, *(p+1), *(p+2));
// Kết quả: 10 20 30
```

★ 5. Truyền Con Trỏ Vào Hàm

Con trỏ cho phép hàm thay đổi giá trị gốc.

Ví dụ:

```
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x=10, y=20;
    swap(&x, &y); // truyền địa chỉ
    printf("x=%d, y=%d", x, y);
    return 0;
}
```

Trước Swap
x = 10
y = 20

Sau Swap
x = 20
y = 10

★ 6. Con Trỏ NULL

Con trỏ không trỏ đến đâu cả. **Cú pháp:**
int *ptr = NULL;
Luôn kiểm tra trước khi dereference.

Ví dụ:

```
int *ptr = NULL;
if (ptr == NULL)
    printf("Con trỏ là NULL");
else
    printf("Giá trị = %d", *ptr);
```

★ 7. Con Trỏ Của Con Trỏ (Double Pointer)

Cú pháp:

kiểu_dữ_liệu **ptr;

Ví dụ:

```
int a = 10;
int *p = &a;
int **pp = &p;
printf("Giá trị = %d\n", **pp); // 10
```

💡 Mẹo Nhanh

- Dùng & để lấy địa chỉ, * để lấy giá trị.
- Luôn khởi tạo con trỏ.
- Kiểm tra NULL trước khi dereference.
- Đặt tên con trỏ có ý nghĩa.

⚠️ Lỗi Thường Gặp

- Dereference con trỏ NULL.
- Dùng con trỏ chưa khởi tạo.
- Quên & khi gọi hàm.
- Số học con trỏ sai kiểu dữ liệu.

LẬP TRÌNH C

10. CẤU TRÚC TRONG C

★ Structure (cấu trúc) là kiểu dữ liệu do người dùng định nghĩa, cho phép nhóm nhiều kiểu dữ liệu khác nhau dưới một tên gọi.

Vì Sao Cần Structure?

- Nhóm dữ liệu liên quan lại với nhau
- Biểu diễn các thực thể trong thực tế
- Cải thiện tổ chức code
- Truyền nhiều giá trị vào hàm

★ 1. Khai Báo Structure

Cú pháp:

```
struct tên_structure
{
    kiểu_dữ_liệu thành_viên1;
    kiểu_dữ_liệu thành_viên2;
    ...
    kiểu_dữ_liệu thành_viênN;
};
```

Ví dụ:

```
struct Student
{
    int roll;
    char name[20];
    float marks;
};
```

★ 2. Khai Báo Biến Structure

Cú pháp:

```
struct tên_structure biến1, biến2, ...;
```

Ví dụ:

```
struct Student s1, s2;
```

★ 3. Truy Cập Thành Viên Structure

Dùng toán tử dấu chấm (.) để truy cập thành viên. **Cú pháp:**
biến.thành_viên;

Ví dụ:

```
s1.roll = 101;
strcpy(s1.name, "Aman");
s1.marks = 85.5;
printf("Tên: %s\n", s1.name); // Aman
printf("Điểm: %.2f\n", s1.marks); // 85.50
```

★ 4. Khởi Tạo Biến Structure

Cách 1: Dùng phép gán

```
struct Student s1 = {101, "Aman", 85.5};
```

Cách 2: Khởi tạo chỉ định (C99)

```
struct Student s2 = {.roll = 102, .name = "Rohit",
.marks = 98.8};
```

★ 5. Structure Kết Hợp Với Hàm

Structure có thể được truyền vào hàm. **Ví dụ:**

```
void printStudent(struct Student s)
{
    printf("Id %-10s %.2f\n", s.roll, s.name, s.marks);
}

int main()
{
    struct Student s1 = {101, "Aman", 85.5};
    printStudent(s1); // truyền structure
    return 0;
}
```

★ Biểu Diễn Trong Bộ Nhớ

Bộ nhớ được cấp cho tất cả thành viên tại các vị trí liên kế.

```
struct Student s1; → s1
```

roll	name[20]	marks
4 bytes	20 bytes	4 bytes

★ 6. Mảng Structure

Ta có thể tạo mảng các structure. **Ví dụ:**

```
struct Student class[3]; // mảng 3 sinh viên
class[0].roll = 101;
strcpy(class[0].name, "Aman");
class[0].marks = 85.5;
// tương tự cho class[1], class[2]
for(int i = 0; i < 3; i++)
{
    printf("Id %s %.2f\n", class[i].roll,
class[i].name, class[i].marks);
}
```

★ Những Điểm Quan Trọng

- sizeof(struct_name) cho tổng dung lượng bộ nhớ của structure.
- Các thành viên được lưu theo đúng thứ tự khai báo.
- Không thêm padding (trong hầu hết trường hợp đơn giản), nhưng alignment có thể thêm padding.
- Structure có thể lồng nhau (structure trong structure khác).
- Dùng typedef để tạo bí danh (alias) cho kiểu structure.

```
typedef struct Student Student; // dùng luôn Student s1;
```

Pro Tip: Structure giúp viết code sạch, module hóa và biểu diễn thực tế tốt hơn.

Ý Tưởng Luyện Tập: Viết chương trình lưu & hiển thị thông tin của N nhân viên bằng structure.

LẬP TRÌNH C

11. UNION TRONG C

★ Union tương tự structure, nhưng nó lưu các kiểu dữ liệu khác nhau tại cùng một vị trí bộ nhớ.

Vì Sao Cần Union?

- Tiết kiệm bộ nhớ
- Lưu một giá trị tại một thời điểm
- Hữu ích khi cần các kiểu dữ liệu khác nhau ở những thời điểm khác nhau

★ 1. Khai Báo Union

Cú pháp:

```
union tên_union
{
    kiểu_dữ_liệu thành_viên1;
    kiểu_dữ_liệu thành_viên2;
    ...
    kiểu_dữ_liệu thành_viênN;
};
```

Ví dụ:

```
union Data
{
    int i;
    float f;
    char ch;
};
```

★ 2. Truy Cập Thành Viên Union

Dùng toán tử dấu chấm (.) để truy cập thành viên union. **Cú pháp:**
tên_union.thành_viên;

Ví dụ:

```
union Data d;
d.i = 10;           // lưu số nguyên
d.f = 22.5;         // ghi đè giá trị trước
d.ch = 'A';         // ghi đè lần nữa
printf("i = %d\n", d.i);
printf("f = %f\n", d.f);
printf("ch = %c\n", d.ch); // kết quả có thể khác nhau
```

★ 3. Biểu Diễn Trong Bộ Nhớ

Tất cả thành viên dùng chung một vùng bộ nhớ.

Kích thước union = kích thước thành viên lớn nhất. **Ví dụ:**

```
union Data
{ int i; float f; char ch; };
```

Thành viên	Kích thước (bytes)	Giá trị lưu tại 1 thời điểm
int i	4	i HOẶC f HOẶC ch
float f	4	i HOẶC f HOẶC ch
char ch	1	i HOẶC f HOẶC ch

Kích thước union Data = 4 bytes

★ 4. Chương Trình Ví Dụ

```
#include <stdio.h>
union Data
{
    int i;
    float f;
    char ch;
};

int main()
{
    union Data d;
    d.i = 100;
    printf("Giá trị i = %d\n", d.i);
    d.f = 3.14; // i bị mất
    printf("Giá trị f = %f\n", d.f);
    d.ch = 'Z'; // f bị mất
    printf("Giá trị ch = %c\n", d.ch);
    return 0;
}
```

Kết quả:

Giá trị i = 100
Giá trị f = 3.140000
Giá trị ch = Z

★ 5. Union vs Structure

Đặc điểm	Structure	Union
Cấp phát bộ nhớ	Riêng cho từng thành viên	Dùng chung bộ nhớ
Kích thước	Tổng tất cả thành viên	Kích thước thành viên lớn nhất
Lưu trữ	Tất cả giá trị cùng lúc	Một giá trị tại một thời điểm
Trường hợp dùng	Nhóm dữ liệu liên quan	Tiết kiệm bộ nhớ, nhiều kiểu dữ liệu khác thời điểm

Mẹo Nhanh

- Union rất mạnh nhưng cần dùng cẩn thận.
- Luôn nhớ thành viên nào đang lưu dữ liệu hiện tại.
- Dùng sai cách có thể dẫn đến kết quả không như mong muốn.

★ 6. Khi Nào Nên Dùng Union?

- Khi bộ nhớ bị giới hạn.
- Khi cần lưu các kiểu dữ liệu khác nhau, nhưng chỉ một tại một thời điểm.
- Dùng trong device driver, giao tiếp dữ liệu, và lập trình low-level.

Lỗi Thường Gặp

- Cho rằng tất cả giá trị đều được giữ nguyên.
- Không kiểm tra thành viên nào đang hợp lệ.
- Bỏ qua kích thước và alignment của kiểu dữ liệu.

LẬP TRÌNH C

12. TIỀN XỬ LÝ TRONG C

★ Chỉ thị tiền xử lý (preprocessor directives) bắt đầu bằng '#' và được xử lý trước khi biên dịch.

Vi Sao Cần Preprocessor?

- Thay thế macro (để đọc hơn)
- Chèn file (file inclusion)
- Biên dịch có điều kiện (conditional compilation)
- Tái sử dụng code

★ 1. #include

Chèn header files vào chương trình. **Cú pháp:**

```
#include <header_file> // header hệ thống
#include "header_file" // header người dùng định nghĩa
```

Ví dụ:

```
#include <stdio.h>
#include "my_header.h"
```

★ 2. #define

Dùng để định nghĩa hằng số hoặc macro. **Cú pháp:**

```
#define TÊN_MACRO giá_trị
#define TÊN_MACRO (biểu_thức)
```

Ví dụ:

```
#define PI 3.1459 // hằng số
#define SQUARE(x) ((x)*(x)) // macro
```

★ 3. Mở Rộng Macro

Preprocessor thay thế macro bằng giá trị của nó trước khi biên dịch. **Ví dụ:**

```
#define MAX 100
int x = MAX; // sau khi tiền xử lý
trở thành →
int x = 100;
```

★ 4. Biên Dịch Có Điều Kiện

Dùng để biên dịch code theo điều kiện.

Cú pháp:

```
#if điều_kiện
// code
#elif điều_kiện
// code
#else
// code
#endif
```

Ví dụ:

```
#define DEBUG 1
#if DEBUG
printf("Chế độ Debug\n");
#else
printf("Chế độ Release\n");
#endif
```

★ 5. Undef

Dùng để hủy định nghĩa một macro. **Cú pháp:**

```
#undef TÊN_MACRO
```

Ví dụ:

```
#define PI 3.14
#undef PI // PI đã bị hủy định nghĩa
```

★ 6. Macro Định Nghĩa Sẵn

Macro	Mô tả
__FILE__	Tên file nguồn hiện tại
__LINE__	Số dòng hiện tại
__DATE__	Ngày biên dịch (VD: "May 18 2025")
__TIME__	Giờ biên dịch (VD: "14:35:20")
__STDC__	Được định nghĩa khi dùng chuẩn C
__cplusplus	Được định nghĩa trong C++

★ 7. Chương Trình Ví Dụ Đầy Đủ

```
#include <stdio.h>
#define MSG "Hello, Preprocessor!"
#define ADD(a,b) ((a) + (b))

int main()
{
    printf("%s\n", MSG); // Hello, Preprocessor!
    printf("Tổng = %d\n", ADD(5, 3)); // 8
    #ifdef DEBUG
        printf("Chế độ Debug\n");
    #else
        printf("Chế độ Release\n");
    #endif
}
```

```
printf("File: %s\n", __FILE__);
printf("Dòng: %d\n", __LINE__);
printf("Biên dịch lúc: %s tại %s\n",
    __DATE__, __TIME__);
return 0;
}
```

Kết quả (khi DEBUG được định nghĩa):

```
Hello, Preprocessor!
Tổng = 8
Chế độ Debug
File: main.c
Dòng: 15
Biên dịch lúc: May 18 2025 tại 14:35:20
```

Tóm Tắt

- #include → chèn header files
- #define → định nghĩa macro/hằng số
- #if, #elif, #else, #endif → biên dịch có điều kiện
- #undef → hủy định nghĩa macro
- Macro định nghĩa sẵn → thông tin hữu ích
- Giúp code sạch, hiệu quả và portable

Pro Tip: Dùng macro cẩn thận, tránh macro phức tạp khó debug.

Ý Tưởng Luyện Tập: Hãy thử viết chương trình dùng #define, biên dịch có điều kiện và macro định nghĩa sẵn.